Adaptive Non-stiff ODE Solver

Problem Formulation

In this document, I describe the implementation of a solver for non-stiff ordinary differential equations. That is, ODEs of the form

$$\mathbf{y}' = \mathbf{f}(t, \mathbf{y}) \tag{1}$$

that satisfy $h\lambda \gg -\infty$.

I will use second-order Adams linear multistep methods, utilizing P(EC)E to reduce computational cost of implicit Adams-Moulton with Adams-Bashforth as the predictor method. I will also use adaptive step size control to obtain a numerical solution to (1) within a target error tolerance. Finally, I will use an interpolating polynomial to approximate the numerical solution at specific time points.

Numerical Schemes

A Linear Multistep Method is a numerical method which involves a linear combination of multiple steps to approximate the solution at the next time step. For non-constant step size Adams methods, we simply integrate (1) across a single timestep of size h_n

$$\mathbf{y}_n = \mathbf{y}_{n-1} + \int_{t_{n-1}}^{t_n} \mathbf{f}(t, \mathbf{y}(t))\, dt$$

The Adams methods approximate this integral using a polynomial interpolation of $\mathbf{f}(t, \mathbf{y}(t))$ using the past values of $\mathbf{f}$. One option for the interpolating polynomial is the Newton interpolation. The Newton interpolating polynomial passing through a set of points at times t_i with the coefficients computed by the divided difference table, defined recursively by

$$\mathbf{f}[t_n, t_{n-k}] = \frac{\mathbf{f}[t_n, t_{n-k+1}] - \mathbf{f}[t_{n-1}, t_{n-k}]}{t_n - t_{n-k}} \tag{2}$$

where $\mathbf{f}[t_n] = \mathbf{f}(t_n, \mathbf{y}(t_n))$. The polynomial is computed with

$$\phi(t) = \mathbf{f}[t_n] + \mathbf{f}[t_n, t_{n-1}](t - t_n) + \mathbf{f}[t_n, t_{n-2}](t - t_n)(t - t_{n-1}) + \cdots + \mathbf{f}[t_n, t_{n-k}](t - t_n) \cdots (t - t_{n-k+1}) \tag{3}$$

For second order Adams-Bashforth, we interpolate the past two $\mathbf{f}$'s. Integrating, we have the Adams-Bashforth method for non-constant step size

$$\mathbf{y}_n = \mathbf{y}_{n-1} + h_n \mathbf{f}_{n-1} \qquad \text{order 1} \tag{4}$$

$$\mathbf{y}_n = \mathbf{y}_{n-1} + h_n \mathbf{f}_{n-1} + \frac{h_n^2}{2h_{n-1}}(\mathbf{f}_{n-1} - \mathbf{f}_{n-2}) \qquad \text{order 2} \tag{5}$$

For Adams-Moulton, we interpolate across the current $\mathbf{f}$ first, and then the past $\mathbf{f}$'s. Thus, the Adams-Moulton method is given by the implicit scheme

$$\mathbf{y}_n = \mathbf{y}_{n-1} + h_n \mathbf{f}_n \qquad \text{order 1} \tag{6}$$

$$\mathbf{y}_n = \mathbf{y}_{n-1} + \frac{h_n}{2}(\mathbf{f}_n + \mathbf{f}_{n-1}) \qquad \text{order 2} \tag{7}$$

In order to avoid the computational cost and complexity of implementation introduced by the implicit Adams-Moulton method, we instead use the Adams-Bashforth method to predict $\mathbf{y}_n$ that is required by $\mathbf{f}_n$ in the Adams-Moulton method. The Predictor-Evaluate-Correct-Evaluate method first predicts $\mathbf{y}_n \approx \mathbf{y}_n^{(0)}$, which is only a good approximation in the non-stiff limit. $\mathbf{f}(t, \mathbf{y})$ is then evaluated at $(t_n, \mathbf{y}_n^0)$. The Corrector method uses this approximation to determine $\mathbf{y}_n$, after which $\mathbf{f}(\mathbf{t}, \mathbf{y})$ is evaluated at $(t_n, \mathbf{y}_n)$. The second-order Predictor-Corrector algorithm is as follows:

$$\begin{aligned}
&\text{Predict} \quad \mathbf{y}_n^{(0)} = \mathbf{y}_{n-1} + h_n \mathbf{f}_{n-1} + \frac{h_n^2}{2h_{n-1}}(\mathbf{f}_{n-1} - \mathbf{f}_{n-2}) \\
&\text{Evaluate} \quad \mathbf{f}_n^{(0)} = \mathbf{f}(t_n, \mathbf{y}_n^{(0)}) \\
&\text{Correct} \quad \mathbf{y}_n = \mathbf{y}_{n-1} + \frac{h_n}{2}(\mathbf{f}_n^{(0)} + \mathbf{f}_{n-1}) \\
&\text{Evaluate} \quad \mathbf{f}_n = \mathbf{f}(t_n, \mathbf{y}_n)
\end{aligned} \tag{8}$$

However, on the first time step, we only have the point $(t_0, \mathbf{y}(t_0))$, so we cannot use a two-step Adams-Bashforth method. In this case, we determine the starting values by using the one-step Adams-Bashforth method as the predictor and the two-step Adams-Moulton method as the corrector.

$$\begin{aligned}
&\text{Predict} \quad \mathbf{y}_n^{(0)} = \mathbf{y}_{n-1} + h_n \mathbf{f}_{n-1} \\
&\text{Evaluate} \quad \mathbf{f}_n^{(0)} = \mathbf{f}(t_n, \mathbf{y}_n^{(0)}) \\
&\text{Correct} \quad \mathbf{y}_n = \mathbf{y}_{n-1} + \frac{h_n}{2}(\mathbf{f}_n^{(0)} + \mathbf{f}_{n-1}) \\
&\text{Evaluate} \quad \mathbf{f}_n = \mathbf{f}(t_n, \mathbf{y}_n)
\end{aligned} \tag{9}$$

For constant step size, the previous equations are still used, but with the additional fact that $h_0 = h_1 = \cdots = h_n$.

Step Size Control

The step size is accepted if the Local Truncation Error is below the provided error tolerance ETOL. The LTE is estimated by the difference between the predictor and corrector methods

$$\text{LTE} \approx \left| \frac{C_{p+1}}{C_{p+1} - \hat{C}_{p+1}} \right| \|\mathbf{y}_{\text{pred}} - \mathbf{y}_{\text{corr}}\| \tag{10}$$

where C_{p+1} is the factor in the LTE estimation of the method

$$\mathbf{d}_n = |C_{p+1} h^p y^{(p+1)}| + \mathcal{O}(h^{p+1}) \tag{11}$$

and $\hat{C}_{p+1}$ for the predictor method and C_{p+1} is the factor for the corrector method.

For Adams-Bashforth, the predictor, $\hat{C}_{p+1} = -\frac{1}{12}$. For Adams-Moulton, the corrector, $\hat{C}_{p+1} = \frac{5}{12}$. Thus, the LTE estimate is

$$\text{EST} \approx \frac{1}{6}\|\mathbf{y}_{\text{pred}} - \mathbf{y}_{\text{corr}}\|. \tag{12}$$

We reject a step if EST > ETOL. Then we adjust the step size by

$$h_n \leftarrow r \cdot h_n \tag{13}$$

where

$$r^{p+1} = \frac{\text{frac} \cdot \text{ETOL}}{\text{EST}} \tag{14}$$

which results in a decrease in step size because EST is larger. If we accept a step, then EST < ETOL and we use (14) as the initialization for the next step size selection iteration, thus resulting in an increase in step size due to smaller value of EST. The algorithm is shown in detail in Algorithm 1.

Algorithm 1 Adaptive Stepsize Update

Require: h_n, $n_{\max}$, ETOL, frac, p

```
h ← h_n
k ← 0
while k < n_max do
    Compute predictor y_pred and corrector y_corr
    EST ← (1/6) ||y_pred − y_corr||
    r ← ((frac·ETOL)/EST)^(1/(p+1))
    h* ← r h
    if EST ≤ ETOL then
        return h*
    end if
    h ← h*
    k ← k + 1
end while
```

Evaluating the Solution at Specific Times

Consider a queue $\mathcal{T}_{\text{eval}}$ containing a sorted list of the times at which to evaluate the solution. Once the simulation time exceeds the value of the first item in the queue, $t > t_{\text{eval}}$, the solution at that time is approximated with a second-order interpolation polynomial $\phi(t_{\text{eval}})$

$$\phi(t) = f[t_n] + f[t_n, t_{n-1}](t - t_n) + f[t_n, t_{n-1}, t_{n-2}](t - t_n)(t - t_{n-1}) \tag{15}$$

where $t_{n-1} < t_{\text{eval}} < t_n$. The algorithm is shown in detail in Algorithm 2.

Algorithm 2 Time Integration with Evaluation-Time Interpolation

Require: initial step size h, final time $t_{\max}$, evaluation-time queue $\mathcal{T}_{\text{eval}}$

```
t_eval ← pop(T_eval)
while T_eval ≠ ∅ or t ≤ t_max do
    Compute next PECE state y_n
    Append new step state to solution history
    while t ≥ t_eval do
        if t_min ≤ t_eval ≤ t_min + h_1 then
            y(t_eval) ← y_{t_0}
        else
            y(t_eval) ← φ(t_eval)
        end if
        if T_eval = ∅ then
            break
        else
            t_eval ← pop(T_eval)
        end if
    end while
end while
```

Examples

Constant h

This example implements the described methods without adaptivity. The IVP is given by

$$\begin{aligned} y_1' &= -y_1 \\ y_2' &= -10(y_2 - t^2) + 2t \end{aligned} \tag{16}$$

with $0 \leq t \leq 1$, $y_1(0) = 1$, $y_2(0) = 2$, and $h = 0.01$. The plot for this example is shown in Figure 1.

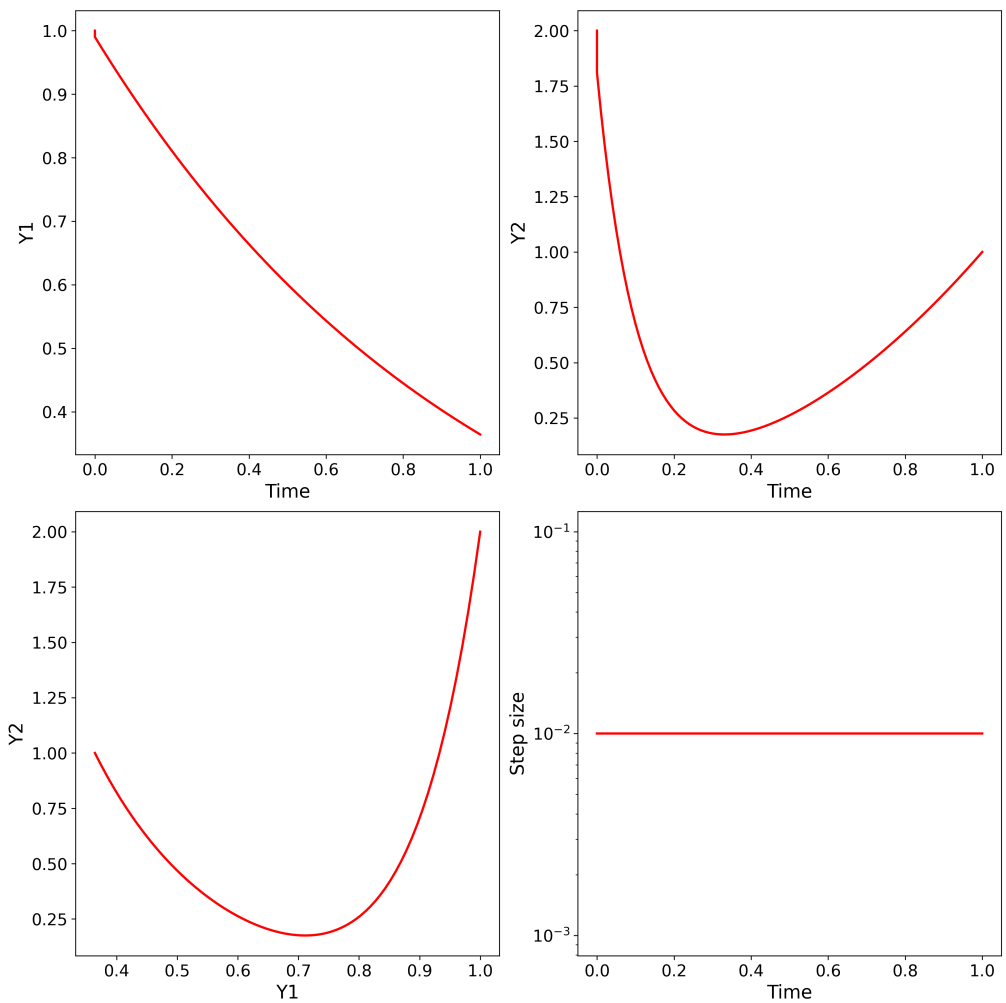

Figure 1: Solution to (16)

Predator-Prey Problem

In this example, I implement the predator-prey problem with adaptivity.

$$\begin{aligned} y_1' &= -0.25y_1 - 0.01y_1y_2 \\ y_2' &= -y_2 + 0.01y_1y_2 \end{aligned} \tag{17}$$

with $0 \leq t \leq 100$, $y_1(0) = 10$, and $y_2(0) = 10$. The plot for this example is shown in Figure 2 for both ETOL $= 10^{-3}$ and ETOL $= 10^{-6}$.

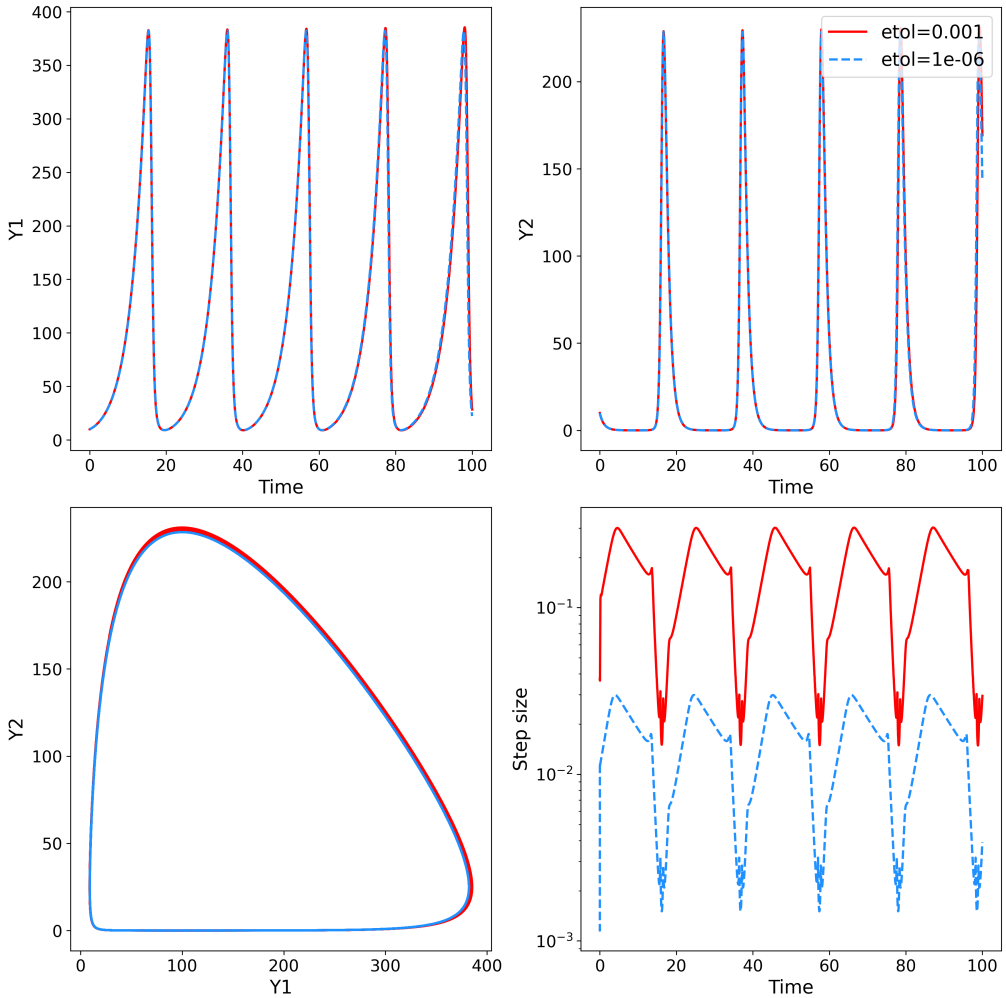

Figure 2: Solution to the Predator-Prey problem (17)

Van der Pol Oscillator

In this example, I implement Van der Pol's equation

$$\begin{aligned} y_1' &= y_2 \\ y_2' &= \eta\left[(1-y_1^2)y_2 - y_1\right] \end{aligned} \tag{18}$$

with $\eta = 2$, $0 \le t \le 11$, $y_1(0) = 2$, and $y_2(0) = 0$. The plot for this example is shown in Figure 3 for both ETOL $= 10^{-3}$ and ETOL $= 10^{-6}$.

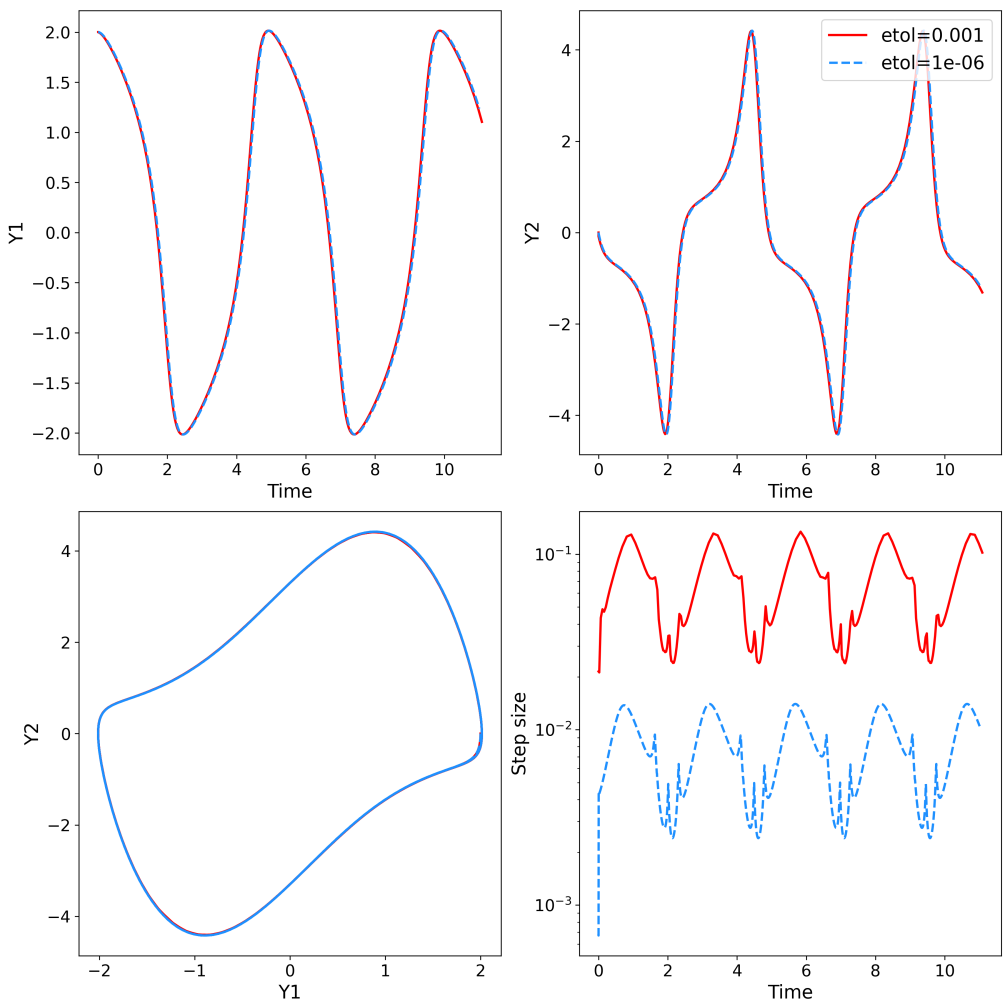

Figure 3: Solution to the Van der Pol Equation (18)

Method of Lines at Specific t

In this problem I solve the advection equation using the method of lines using the backward difference (upwind scheme).

$$u_t + u_x = 0, \qquad u(0,x) = e^{-10x} \tag{19}$$

for $x \in [0,1]$ with boundary condition $u(t,0) = 1$, evaluated at $t = \{0, 0.25, 0.5, 0.6, 0.8, 1\}$ using interpolation. There are k spatial steps, with $\Delta x = 1/k$. The backward difference scheme is

$$\frac{du_i}{dt} + \frac{u_i - u_{i-1}}{\Delta x^2} = 0 \tag{20}$$

which, in the standard ODE form,

$$\mathbf{u}' = A\mathbf{u} \tag{21}$$

where A is an $(k+1) \times (k+1)$ lower triangular matrix with $a_{ij} = \frac{1}{\Delta x^2}(\delta_{(i-1)(j-1)} - \delta_{ij})$ and $a_{11} = 1$. The plot for this example is shown in Figure 4.

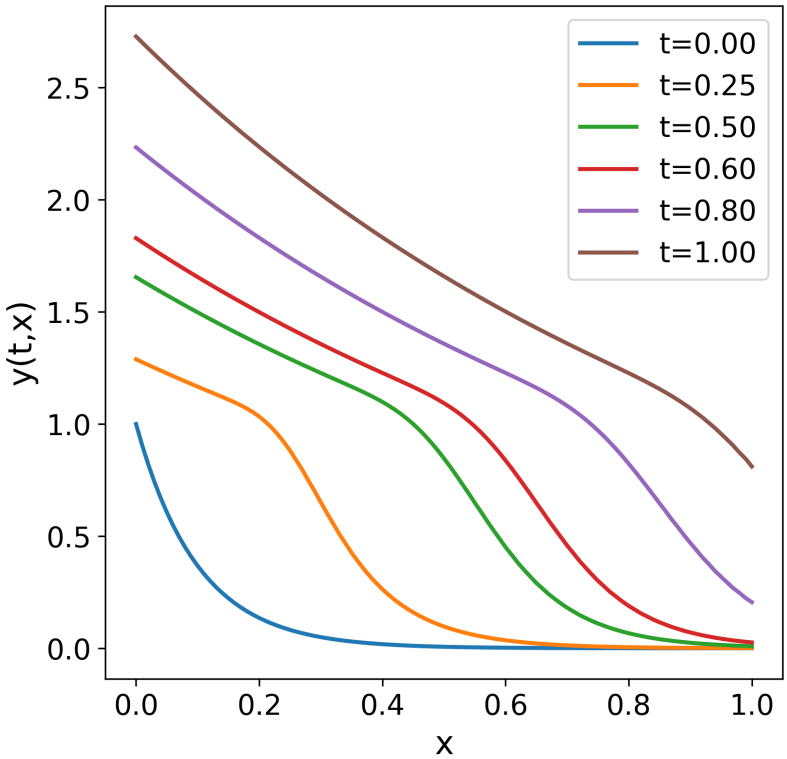

Figure 4: Method of Lines solution to (19)